BrewFS: Delta-Based Multi-Agent Workspaces with Packed Metadata and Variable-Granularity Object Storage

Internal research draft
23 October 2026

Abstract

Coding agents need ordinary writable volumes, isolated views of shared environments, and read-only access to published state. We present the design of BrewFS, an object-backed file system with three optional, composable mechanisms. Delta-based workspaces resolve a shared immutable baseline and private changes into a task's file-system view. Packed Metadata organizes immutable directory entries, attributes, and data locators into indexed pages on object storage; it can serve a workspace lower or a standalone read-only mount. Variable-Granularity Object Storage separates client object formation from optional backend encoding and verification layouts. A workspace can use native metadata, a packed snapshot can operate without private changes, and an ordinary volume can choose object granularity independently of both. We explain their state, access, and publication rules, then distinguish architectural composition from combinations supported by current mount paths. A reanalysis of 1,138,884 FUSE requests from eight coding-agent task runs characterizes metadata traffic, read/write sizes, and temporal heterogeneity. The evaluation plan tests each mechanism and their legal combinations while controlling caching, request coalescing, durability, and construction costs. The workload observations do not establish system speedups.

Status. This is an author working document: design, source audit, and measured passthrough workload observations are distinguished from planned system experiments. Integration, correctness, and performance claims require the evidence described below.

1 Introduction

Coding agents navigate repositories, read and edit files, and invoke builds and tests through repeated tool-use loops [21, 24]. Several tasks may begin with the same repository and dependencies while requiring independent subsequent changes. Other tasks need only an ordinary writable volume, or read-only access to a published environment. A file system serving these uses must distinguish the semantics of a task's view from the representation of its metadata and the geometry of its stored data.

These choices address different costs. Copying a baseline for every task duplicates provisioning work and private state. Sharing that baseline leaves open how clients obtain its directory entries, attributes, and data locations. Storing those records in objects introduces a request-versus-transfer tradeoff. The data path has a related but separate tradeoff: larger objects can amortize requests, while smaller independently readable units can avoid fetching unused bytes. A short application read may still cause a much larger object transfer or physical storage read.

BrewFS is an object-backed file system that exposes three optional mechanisms at these boundaries. *Delta-based multi-agent workspaces* compose a shared immutable baseline with private namespace and data-mapping state. *Packed Metadata* organizes immutable records into indexed, bounded groups on object storage. *Variable-Granularity Object Storage* controls client object formation and, on a compatible backend, permits optional encoding and verification layouts. Each mechanism has a useful role without the other two: a workspace can inherit a native metadata layer, a packed snapshot can be mounted read-only, and an ordinary volume can choose its object block size while using the backend's default layout.

The mechanisms also compose. Many workspaces can reference one packed baseline, amortizing its construction and reusing fetched pages. Their private changes leave immutable baseline objects available to other readers. A packed container or a newly uploaded data object may additionally use a compatible backend layout. These interactions do not imply additive or multiplicative gains. Frequent publication can erase packing's amortization, unrelated point lookups can waste page bytes, and whole-object verification can prevent a small file read from becoming a small backend range. Selection should follow the workload and persistence contract.

The individual ideas have substantial precedents. OverlayFS provides layered namespace semantics [12],

BranchFS uses copy-on-write branches for agentic exploration [20], and DeltaFS separates private namespace changes over shared storage [26]. Immutable distribution systems already support indexed, demand-loaded baselines [2, 14]. Our research question is therefore: *where should view composition, immutable metadata access, and object granularity be selected, and under which workloads do their individual or combined costs justify those choices?* Full directory copying is one control, not the only relevant baseline.

This paper develops the state and access rules behind these choices. It separates a workspace's durable private state from transient dirty buffers, and a baseline's logical identity from its metadata format. It also separates the client object block size g, finalized object length L_o, backend erasure-coding block size B, and physical verification/read quantum q. The current implementation supports concrete subsets through different mount paths; architectural optionality does not mean that every combination is available as interchangeable runtime flags.

The evidence consists of an implementation audit and a reanalysis of 1,138,884 FUSE requests from eight coding-agent task runs. These observations characterize request structure, rather than BrewFS's performance. We specify controlled experiments for each mechanism, legal combinations, lifecycle costs, and complete tasks. The resulting contribution is a design and research protocol whose hypotheses can be tested without confusing a change of runtime, cache policy, or durability boundary with a mechanism's effect.

2 Background and Motivation

2.1 The ordinary file-system path

Tools issue file operations through FUSE. The file system resolves names and attributes through its metadata path, translates file offsets through data mappings, and obtains payload bytes from local caches or object storage. Writes introduce new data and update those mappings under the selected visibility and durability protocol. This ordinary path is useful without shared workspaces, packed snapshots, or backend layout hints.

A workspace can contain source files, dependency directories, generated artifacts, build trees, and temporary files. Some subtrees remain stable across tasks; others are frequently rewritten. A deployment can choose stable state as a shared baseline while retaining private changes for each task. The usefulness of doing so depends on cross-task overlap, mutation footprint, and baseline lifetime. Agent framework papers establish that repository navigation and editing occur; they do not establish that every task shares most of its state or benefits from a layered view.

2.2 Observed requests in real agent runs

Capture and scope. We reanalyzed eight SWE-bench Verified task runs [6]: five Core and three Exploratory. All used DSH 0.1.5-rc.1 and the CCSwitch-configured model `deepseek-v4-flash`. Exploratory runs used a broader investigation/testing prompt and a 30-minute budget; all finished earlier. Each base-commit checkout was mounted through `libfuse-fs PassthroughFs` on Ubuntu 24.04 under WSL; `rfuse3 LoggingFileSystem` recorded actual FUSE handler requests. Streaming reanalysis covered all 1,138,884 events: 217,129 Core and 921,755 Exploratory. Counts retain every status, 64 setup/canary requests before agent startup, and teardown, including 24 requests after recorded run finish. Group statistics weight requests equally, not tasks. Per-task counts and source-validation details are in the companion workload record.

Metadata traffic and read dominance. The six metadata methods accounted for 61.5% of Core and 56.4% of Exploratory calls (Figure 1a). Restricting the denominator and numerator to `ok` events reduces these shares to 52.0% and 44.2%: failed probes materially change the request mix. Across tasks, the all-status share ranged from 39.6% in SymPy to 65.4% in Django, so the group majority does not describe every task. Read calls outnumbered writes by 19.9:1 in Core and 4.16:1 in Exploratory; per-task ratios ranged from 3.12:1 (Pytest) to 42.7:1 (Matplotlib). These observations motivate evaluating both positive and negative metadata paths alongside data I/O.

A mixture of request sizes. Reads ranged from 4 to 128 KiB, with Core and Exploratory medians of 8 and 16 KiB. Writes ranged from one byte to 128 KiB, with medians of 4,084 and 4,117 bytes, respectively. Requests of at most 4 KiB accounted for 47.7% and 34.5% of reads, and 74.3% and 49.9% of writes. The interval $(4, 64]$ KiB contained a further 36.2% and 36.4% of reads and 21.3% and 44.0% of writes (Figure 1b). This mixture motivates measuring how client fetching and uploads transform requests before testing backend layouts. FUSE observations occur below kernel caching and readahead; they do not identify application syscall sizes, unique bytes, finalized object lengths, or an optimal block size.

Task and temporal heterogeneity. Tasks generated 8,542–634,726 calls. Astropy contributed 68.9% of Exploratory calls, versus 6.0% for Pytest. Capture spans, from first to last request start, were 42.7–71.3 seconds for Core and 119.9–556.1 seconds for Exploratory. The busiest 10% of one-second windows, rounded up per task, contained 47.4–83.7% of its calls (Figure 1c). The companion record retains the full method breakdowns, size ECDFs, and per-task cumulative time curves. Replay should retain these per-

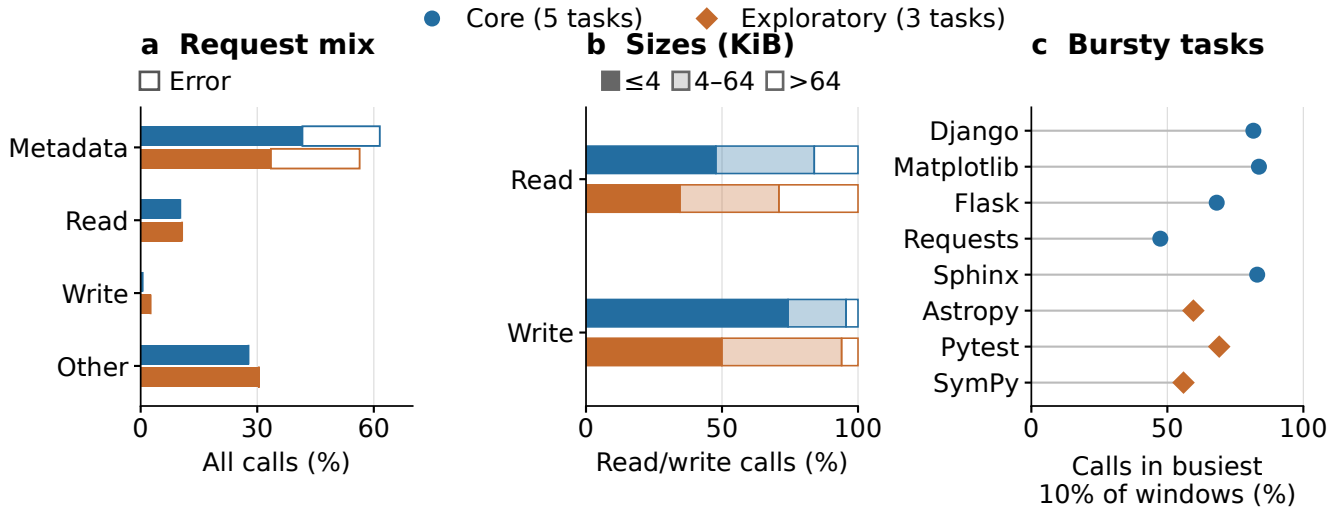


Figure 1: Request structure across eight single-run FUSE captures. Core bars precede Exploratory bars in each pair. (a) Shares of all calls; white segments are errors within each total. Metadata comprises `lookup`, `getattr`, `getxattr`, `readdirplus`, `opendir`, and `releasedir`; Other contains every remaining method, including file open/flush/release. (b) Each bar includes all reads or writes in its group. Dark, light, and white segments denote ≤ 4, $(4, 64]$, and > 64 KiB, respectively (1 KiB = 1024 bytes). Read counts are 22,142/98,292 and write counts 1,112/23,600 (Core/Exploratory), all `ok`. Lengths are requested FUSE bytes, not object sizes. (c) Fraction of each task's calls in its busiest 10% of one-second windows, rounding the window count up; zero-call and final partial windows are retained. All panels retain setup/teardown and every recorded status.

task mixes and bursts. Each task was observed once; different repositories, prompts, and budgets prevent a causal interpretation of the group contrast. Without tool-phase labels, bursts cannot be assigned to searching, editing, or testing.

Status semantics and evidence limits. The capture contained 872,519 `ok`, 253,669 `error`, and 12,696 `incomplete` events. Errors occurred in `lookup` (233,452), `getxattr` (20,203), and `rmdir` (14). Collection notes associate many with failed path probes or unsupported attributes; absent errno values prevent event-level cause attribution. The parser labels a request `incomplete` when no recognized completion is paired with it: here, 12,688 `forget` and eight `destroy` records. Neither label counts failed agent tasks. All eight agent and daemon exit codes were zero, which does not establish SWE-bench grading success.

Handler latency measures WSL passthrough intervals and excludes model response time. Elapsed capture time instead includes inter-request gaps, which may contain model waiting; summing overlapping handler intervals cannot recover task wall time. These observations do not measure RustFS, Redis, network object storage, or cloud-disk latency. They motivate metadata and mixed-I/O experiments for this single-agent/model panel, but do not establish cross-task sharing, sparse mutation, or performance gains from BrewFS, RustFS, or the automatic hint policy. Those remain questions for Section 8.

2.3 View sharing and metadata access are separate choices

A shared view needs a stable identity and rules for resolving private changes. It does not prescribe how baseline records are stored. A native catalog or KV namespace can supply the same immutable names and attributes as an object-resident packed representation, provided each satisfies the view's lookup contract. Conversely, a read-only consumer can use packed metadata without allocating a writable workspace.

Metadata organization and caching already reduce service costs in TableFS, IndexFS, LocoFS, and InfiniFS [11, 13, 17, 18]. It would be misleading to assume that every operation on native metadata requires a separate network round trip. Packing exploits a more specific property: records that cannot change after publication can be indexed and fetched together without joining the mutable update protocol. A directory scan may reuse names, attributes, and data locators from one fetched page; a random point lookup may consume only a small part of that page.

The relevant property is *lower-query independence*: supported baseline queries can obtain their records from the snapshot objects. Mutable upper records, reader pins, publication, and reclamation may still use a metadata service. DeltaFS's private logs and selectively assembled namespaces likewise show why namespace sharing must be compared with more than one global mutable metadata design [26]. Equal cache budgets and appropriate batch access belong in

these comparisons.

2.4 Object formation and physical work have different boundaries

An application's request size, a logical chunk, the client block size g, and the finalized object length L_o describe different stages. A chunk provides an addressing range; slice mappings describe visible file data; blocks determine how a slice becomes objects. Tail blocks, compression, and framing can make L_o differ from g. An object format and the client's cache and verification policy then determine whether a read fetches a range or an entire object.

Backend coding adds further boundaries. An erasure-coding block of size B is distributed across data and parity shards. The physical verification/read quantum q determines the verified units required within the serving shards. Storage systems already make different choices about striping, coding, and redundancy transitions [1, 5, 8]. A compatible backend can expose an additional layout choice without changing the file system's view semantics or the client's object boundaries.

These distinctions motivate separate measurements of requests, network bytes, physical bytes, CPU work, and durable write completion. A smaller g can increase request count; a smaller q adds checksums and processing; a fixed data/parity ratio retains its nominal redundancy. Caches and whole-object verification can hide a physical-layout benefit. The evaluation must trace actual object formation and reads rather than infer storage work directly from a FUSE request histogram.

3 Overview and System Contract

3.1 Three optional mechanisms over a common file interface

The common path provides file operations, metadata resolution, and object access. Three choices refine that path at different boundaries (Table 1). View composition determines which state a task sees. Metadata representation determines how immutable records are located and fetched. Object granularity determines how payloads are formed and how much work their access requires. No choice semantically requires the other two.

For example, a task requiring independent edits can use a delta-based workspace over a native immutable layer. A read-only environment can use a packed snapshot directly. An ordinary writable volume can configure its object block size without either facility. When repeated tasks share a stable environment, they can instead combine workspaces and a packed baseline. Optional backend layout selection is another data-path decision in each eligible uploader, rather than a condition for any of these view types.

3.2 A representation-independent workspace contract

Let S identify a shared immutable baseline and Δ_i the private writable state of workspace i. Its view is

$$W_i = \text{Resolve}(S, \Delta_i). \tag{1}$$

The baseline exposes name, attribute, and data-location queries. It may bind a sealed native layer or a packed manifest; S does not mean that every workspace has a packed lower. Clients pin an identity instead of following a mutable "latest" pointer on each lookup. A selected subtree can form a baseline without freezing an entire source volume.

The upper represents private names, attributes, deletion state, and data mappings. Newly written payload bytes reside in separately referenced objects. Resolution checks private state first and inherits from S only when the upper permits it. A deletion marker suppresses a lower entry, while absence of an upper record permits inheritance. This contract leaves the lower's physical metadata representation open.

The isolation property is view-local mutation: an operation in W_i leaves S and another workspace's private state unchanged. This property does not provide process isolation, network restrictions, or automatic merging of independent edits. Those functions require their own runtime or application protocols. Concurrent operations within a workspace must separately preserve its namespace and data invariants.

3.3 Composition through explicit interfaces

Figure 2 illustrates one possible composition using an explicit upper data map, a packed lower, and optional backend layout hints. The figure expands the interfaces between mechanisms; it is not the minimum configuration. For example, two workspaces can share a fetched baseline page, while one records a private overwrite and the other continues to read S. The chosen lower reader resolves inherited data locators to payload ranges. The backend serves those objects using their persisted storage layout.

Composition preserves three boundaries. First, transient dirty buffers may provide local read-after-write behavior before objects are remotely durable; they are distinct from the workspace's private namespace state. Second, publication makes a validated immutable version visible only after its required objects satisfy the selected persistence contract. Reader ownership must keep referenced state alive through reclamation. Third, logical object identity and file contents remain independent of the backend's coding layout. Changing a backend setting does not change an already published view.

A recoverable file version can include explicit application checkpoints, but does not reconstruct process memory, open handles, or execution state by itself. Section 7 distinguishes current mount paths and recovery support. The following

Choice	Mechanism selected	Alternative retained
View composition	Delta-based workspace: shared baseline plus private state	Ordinary volume with its own namespace and mappings
Immutable metadata	Packed Metadata: indexed records in bounded object-resident pages	Native catalog or KV representation
Object granularity	Configured client block size g; optional compatible backend B/q choices	Fixed client geometry and backend default layout

Table 1: Independent design decisions. A deployment selects mechanisms for their own costs and contracts. Current mount paths support specific combinations; this table is not a claim of eight interchangeable runtime modes.

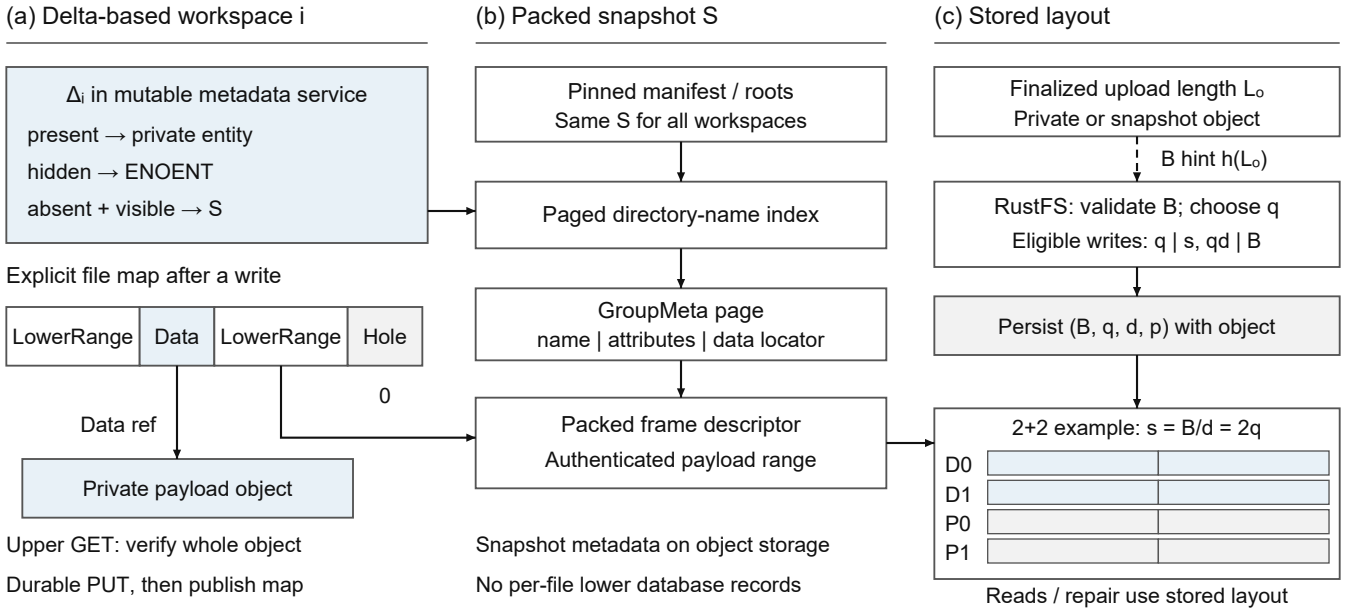


Figure 2: An illustrative combination of the optional mechanisms. (a) An Entity-style upper resolves private state and explicit Data, LowerRange, and Hole intervals. (b) A packed lower resolves a representative name lookup through an index and GroupMeta page to a payload descriptor. A native lower can implement the baseline contract instead; Packed also supports standalone read-only use. (c) Finalized objects may request a compatible backend layout; omitted hints retain default selection. Client object formation with g precedes this panel and is detailed in Section 6. Solid arrows show access dependencies within this combination; dashed hint wiring remains incomplete in the illustrated Entity/producer paths. The 2+2 shards are schematic. This is a design composition, not an end-to-end result.

sections develop the principles of each mechanism before those implementation-specific limits.

4 Delta-Based Workspaces

4.1 Shared baseline, private state

Delta-based workspaces separate a task's modifications from the state it shares with other tasks. A workspace binds a shared immutable lower S and maintains a private writable upper Δ_i; its view is $W_i = \text{Resolve}(S, \Delta_i)$. The lower may be a sealed catalog in the metadata database or a packed snapshot on object storage. These choices change how inherited records are retrieved, while the workspace contract determines which records are visible. Neither Packed Metadata nor a particular object-storage layout is required by this contract.

Creating a workspace registers its base identity and independent private state. The catalog path identifies the base by its layer, sealed version, and root hash; a packed lower additionally binds its authenticated manifest. Unchanged entries and file contents remain reachable through this identity, so creation need not copy the complete metadata tree. Binding validation, ownership records, and mount preparation still have costs.

A private name record distinguishes a present entry, an

explicit deletion, and absence. For parent directory d and name n,

$$R_i(d,n) = \begin{cases} e, & \Delta_i(d,n) = \text{Present}(e), \\ \perp, & \text{the name is hidden in } W_i, \\ R_S(d,n), & \text{otherwise.} \end{cases} \quad (2)$$

A whiteout suppresses the inherited name; applicable directory-opacity rules can also prevent inheritance. Absence permits fallback only when no such rule hides the name. Removing an upper payload therefore does not undo a deletion marker and resurrect a lower file.

Directory enumeration applies the same precedence to each name, removes duplicates, and returns attributes of the visible inode. Private attributes describe its current state; they are not an arithmetic merge with lower attributes. Hardlink aliases must continue to identify the same logical inode when private state is materialized. Namespace mutations such as rename must update related entries consistently within the workspace.

4.2 Changed data and local dirty reads

The catalog path records private data extents alongside namespace and inode changes. The Entity path supplies one concrete current-state representation: each changed logical chunk has a normalized map with explicit *Data*, *LowerRange*, and *Hole* spans. Data references newly stored upper bytes, LowerRange references inherited bytes in S, and Hole reads as zeros. An uncovered interval in an explicit map is not an implicit lower reference. The map must describe the applicable range completely.

A short overwrite stores new bytes and replaces their mapped interval; unchanged intervals can keep their lower references. This requires neither a byte-difference algorithm nor a copy of the whole lower file. Truncation needs an additional inheritance boundary. Entity maintains a lower-visible size that can decrease but cannot increase. If a truncated file is extended, the newly exposed region contains zeros or new upper data, rather than old bytes resurrected from S. Object formation and integrity checking remain separate choices: a short logical write need not produce an equally short object.

The VFS dirty overlay serves a different purpose: reading this client's bytes before writeback completes. If local dirty data covers $[4,8)$ KiB, a read of $[2,10)$ KiB combines two committed 2 KiB ranges with the dirty 4 KiB range. Workspace delta state instead determines persistent namespace, attribute, and data-mapping visibility. A buffered workspace path can use both mechanisms; local read-your-writes behavior alone establishes neither remote durability nor a published recovery point.

4.3 Publication and workspace lifetime

Data publication must respect the selected durability boundary. The Entity path uploads an object durably before publishing its mapping. The ordinary VFS offers upload-before-commit and explicitly selected commit-before-upload modes; the latter can expose a mapping before its object is remotely available. Consequently, preparing a recoverable version requires an explicit drain and validation boundary, even when local reads already return new data.

A worker handoff starts with application checkpoints and a stop to new source writes. Required payloads and metadata must become durable and validated before an immutable version reference is published. The destination then binds that version and acquires authority for subsequent private writes. Lease and generation checks must reject stale writers. The handoff protocol must establish a file recovery point. The application runtime is responsible for restoring sessions, restarting tools, and reopening files. The file version does not transfer running processes or guarantee recovery of progress beyond the last confirmed publication. The available publication and recovery paths are distinguished in Section 7.

Lifetime management applies to either lower representation. A workspace or reader holding S must retain its referenced metadata and data. Publishing a replacement must not invalidate existing readers, and reclamation must separate live references from abandoned construction objects. For one shared baseline, live logical metadata obeys

$$M_{\text{live-meta}} = M_S + \sum_i M_{\Delta_i} + M_{\text{control}}. \quad (3)$$

M_S counts the shared metadata once; M_{Δ_i} counts private names, attributes, deletion records, and mappings; M_{control} counts disjoint binding and ownership records. Upper payload, encoding and checksum overhead, unreclaimed objects, and caches belong to separate physical accounting. Private state includes complete current records and mapping overhead, so its cost grows with modification and deletion. The evaluation will measure this cost against materialized workspaces under matched visibility and durability contracts.

5 Packed Metadata

5.1 An optional representation for immutable state

Packed metadata changes how fixed file-system records are stored and retrieved. Its input is a selected source view containing directory entries, attributes, and file-data mappings. Its output is a packed immutable snapshot: authenticated records and access structures that describe the selected subtree and locate its contents. Freezing these records permits the reader to reuse their organization and validation across requests.

The same snapshot has two uses. An independent read-only mount serves it directly through the file interface. A delta-based workspace can instead bind it as its shared immutable lower, with private state resolved by the workspace mechanism. Packed metadata therefore does not require delta-based workspaces; a shared workspace baseline can likewise use native catalog records. It also operates with the object backend's default layout. These choices separate immutable access organization from view composition and backend encoding.

5.2 Freezing records into indexed, bounded units

The organizing principle is to move work from repeated record lookup to snapshot construction. The producer groups fixed records, encodes bounded pages, and builds range indexes over their keys. A reader pins the manifest identity and follows authenticated references to the required pages. The index describes ordered metadata ranges; its height follows page capacity and record count, independently of the source directory tree's depth.

In the current representation, a logical read group covers a parent directory and a range of raw entry names in byte order. Its GroupMeta page co-locates directory entries, common attributes, and file-data locators. This organization lets a name lookup return information needed by later attribute and data operations. The raw page is bounded at 256 KiB, while a physical container may hold multiple groups. A group, a page, and an object thus have different roles and sizes.

Metadata co-location, content inlining, compression, and data co-packing are separate choices. Co-location combines related records; inlining puts small file contents in a metadata page; compression changes the encoded bytes; co-packing places multiple data regions in a container. Each can change transferred bytes, decoding work, or cache use independently. The layout comparison must distinguish their contributions.

5.3 Fetching, verifying, and reusing records

For `lookup(parent,name)`, the reader follows the directory-name index to the group covering that key, resolves the container reference, and fetches the GroupMeta byte range. It verifies the stored-byte digest before decoding, then checks the resulting entry count and name boundaries against the authenticated group reference. The requested record supplies the inode, common attributes, and data locator. Figure 3 shows this path.

Other operations use the index appropriate to their input. Direct `getattr` can obtain common attributes through an inode index. Enumeration traverses ordered ranges, with counted indexes supporting directory-ordinal navigation. A file read uses inline contents or resolves a descriptor for the referenced data region, fetches the applicable range, and verifies it. Packed data frames provide snapshot-level access and integrity units; the backend interprets these object requests using its own persisted layout.

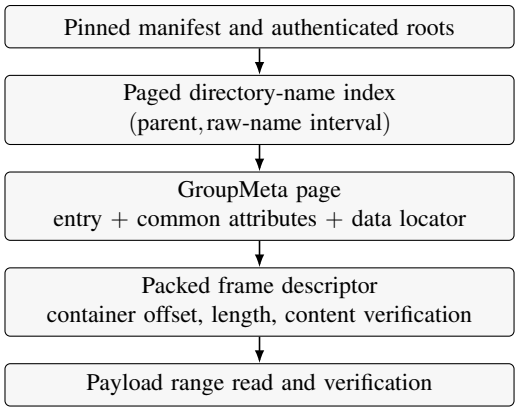


Figure 3: A representative packed-snapshot access, used directly by a read-only mount or after workspace resolution selects the lower. A name lookup obtains an entry, common attributes, and a data locator. Referenced payloads use authenticated descriptors and ranges. Other operations may use separate indexes; the path does not imply a fixed GET count.

Authenticated pages and resolved records can be cached under the pinned snapshot identity. Immutability makes reuse valid for that identity without tracking per-record updates. Bounded pages allow demand loading, but a cold lookup may traverse several index pages, and a data read may need additional descriptor and payload requests. Cache state and access locality therefore affect both request count and useful bytes per fetch. Failed authentication or a missing referenced object is an error, distinct from an authenticated absence result.

For supported snapshot queries, these structures provide the required namespace, attributes, and data locations without consulting per-file records in the mutable metadata database. In a writable workspace, private upper checks and control operations remain separate database traffic. Cold-access tests must distinguish those requests from lower-record lookups when establishing lower-query independence.

5.4 Construction, publication, and reuse cost

Construction fixes a source view, builds and uploads records and data objects, and produces the indexed manifest. Source consistency requires a snapshot or an explicit validation protocol. An immutable output alone does not make a changing input atomic. Before publishing a new binding, the selected protocol must establish that referenced dependencies are durable and valid. Producing a manifest and publishing its reference are separate events. Existing readers retain their

pinned view until its objects can safely be reclaimed; ordinary file close does not rebuild it.

The current concrete format uses PM11 manifests, IP06 indexes, and GroupMeta pages. It instantiates the frozen-record and indexed-fetch principles with directory-oriented joint records. Earlier tree-and-batch formats express the same organizing principle through different record and wire layouts; their compatibility and mount support are implementation properties.

Construction, validation, publication, rebuilding, and reclamation all enter the cost comparison. For each additive resource metric c, we will measure

$$C_c(K) = C_{c,\text{build}} + C_{c,\text{publish}} + \sum_{i=1}^{K} C_{c,i} + C_{c,\text{reclaim}}. \quad (4)$$

CPU-seconds, transferred bytes, and monetary cost are separate metrics; concurrent task wall-clock times are not summed into a batch makespan. Reuse pays for construction only when the accumulated access savings exceed these costs. Frequent rebuilding, weak name locality, or expensive cold index walks can erase the benefit.

Immutable object-backed metadata already has strong precedents [2, 14, 26]. We will compare the joint layout with immutable sorted-KV records at equal content and cache budgets, and with compatible demand-loaded immutable-image systems. Standalone read-only and workspace-lower use are separate comparisons. The sorted-KV control tests layout organization and is not a reproduction of DeltaFS.

6 Variable-Granularity Object Storage

6.1 An independent choice of object granularity

Object granularity controls how file data becomes uploadable objects and how much work a subsequent access requires. It is useful in an ordinary flat volume as well as in a workspace or an immutable snapshot; neither delta-based workspaces nor Packed Metadata is a prerequisite. A deployment can retain the fixed client block size and the backend's default layout. Selecting another granularity changes this data path without changing the namespace's inheritance or deletion rules.

There are several distinct boundaries. Let g denote the client block size, and L_o the finalized upload length after applicable compression, headers, and framing. The client then chooses which object ranges to fetch. On a compatible backend, B denotes the erasure-coding block size and q the physical verification/read quantum within a shard. These quantities describe different work: object formation, transferred bytes, encoding, and physical verification. Changing one does not necessarily change the others.

g	Object count	Whole-object bytes fetched
1 MiB	64	2 MiB
4 MiB	16	4 MiB
8 MiB	8	8 MiB
16 MiB	4	16 MiB

Table 2: Idealized object formation and a 2 MiB partial read. The last column assumes complete fetches of the intersecting objects, with no cache hits or compression; it does not describe every range-read path.

6.2 Forming objects from file updates

The ordinary data path locates a file offset within a logical chunk and records written content in slices. A chunk is an addressing range, rather than a remote object. The uploader partitions each slice's data into blocks using g and stores them under the slice identity and block index. Thus a 10 MiB slice with $g = 4\,\text{MiB}$ can produce raw blocks of 4, 4, and 2 MiB. Compression and framing subsequently determine each L_o; a short tail need not occupy a full block. Flush boundaries and partial writes also affect the objects actually formed.

The reader resolves the visible slice intervals and translates them into object offsets using the same geometry. A new slice can cover an old interval while untouched intervals still refer to older objects. Such an update need not rewrite the complete old object. Compaction, upload retries, and the accumulation of short slices can nevertheless add work beyond the newly written bytes.

The present ordinary path defaults to 64 MiB chunks and $g = 4\,\text{MiB}$. Its client block size is a fixed configuration, with a chunk size that is a nonzero multiple of g. The proposed 1/4/8/16 MiB choices express a design space for object formation; an online controller that selects among them from access behavior is not implemented in this path. Readers currently reconstruct block addresses from the configured geometry, so changing g for existing data requires a compatible representation or migration.

6.3 Trading request count against transferred work

Larger objects can reduce the number of requests needed to transfer a long contiguous slice. Smaller objects can reduce excess fetching when decoding, authentication, or cache policy requires a complete object. Table 2 illustrates this trade-off for one uncompressed, aligned 64 MiB slice and a read of file interval $[10, 12)\,\text{MiB}$. These are arithmetic examples, not measurements or a policy recommendation.

Range GET can retrieve only part of an object when its format permits independent access. The ordinary store supports page-aligned range reads for eligible small requests, while

compressed blocks or a whole-object fetch policy can require larger transfers. A packed frame supplies another decoding and integrity boundary inside a snapshot object. Its size is neither g nor the backend's q. Caching and concurrent-read coalescing further change how many remote requests reach the backend.

We therefore separate network read amplification, upload amplification, and physical read amplification. Their numerators are, respectively, object response bytes, uploaded bytes, and storage-device bytes; their denominators are the corresponding application bytes served or written. Request counts, compression work, and cache occupancy remain separate costs. Smaller objects do not guarantee lower write amplification, and a large object does not imply a complete download for every small read.

6.4 Optional backend layout

A compatible RustFS research backend provides a further choice after the object has formed. The client may send an encoding-block-size hint $h(L_o)$ based on the finalized upload length. This interface leaves g and multipart part sizes unchanged. Omitting the hint retains the backend's default selection. The backend validates an enabled request and persists the effective B; an emitted header alone does not establish which layout was used. Multipart initiation uses the complete object's length, and retries retain that object's decision.

The current hint modes are disabled, fixed, and automatic. Fixed candidates are 64 KiB, 256 KiB, 1 MiB, and 4 MiB. The automatic hint policy omits the hint for $L_o \leq 1\,\mathrm{MiB}$ and requests $B = 1\,\mathrm{MiB}$ otherwise. This is a threshold rule rather than access-adaptive object formation. Because the inspected backend's default B is also 1 MiB, this rule may leave the effective encoding block size unchanged.

For d data shards and p parity shards, let s be one complete stripe's logical shard size. A separately enabled backend policy can select a physical verification/read quantum subject to

$$q \mid s, \qquad qd \mid B. \tag{5}$$

These constraints preserve stripe alignment while permitting smaller verification units. Both hint admission and new q framing have their own compatibility gates. Reads and healing use persisted object geometry even after write settings change.

For a nonempty shard interval starting at a with length ℓ, the number of verification/read quanta intersected is

$$n_q(a,\ell) = \left\lceil \frac{a+\ell}{q} \right\rceil - \left\lfloor \frac{a}{q} \right\rfloor. \tag{6}$$

Ignoring tails and batching, the covered data is approximately $n_q q$ plus checksums, summed over shards actually read. The application request cannot be substituted directly for ℓ: the client range, compression, and coding determine that interval. Smaller q can reduce over-read while increasing checksum overhead. Fixed (d,p) still has nominal redundancy $(d+p)/d$; a large B does not imply a fully padded tail stripe.

6.5 Composition and scope

A workspace changes which objects a view references; Packed Metadata changes how immutable records and payload locations are obtained. Either can use default backend layouts. Conversely, a flat volume can configure g and eligible backend hints without either mechanism. Snapshot producers and workspace upper writers retain their own object formats and upload contracts, so one path's block geometry cannot be assumed for all paths. Section 7 identifies the current integration limits. It also records the ordinary path's coupling between enabled hints and whole-object caching, which must be held constant when isolating B or q.

7 Implementation and Integration Status

The source audit fixes BrewFS at `242af5f7e9bb` and the RustFS research fork at `cc4926fac110`.[1] These revisions define the implementation scope below. Source inspection establishes which components and connections exist; it does not establish successful deployment or measured system behavior. This revision adds no mount, fault-injection, or performance experiments.

Mount entry points. Table 3 separates the current paths. Their separation realizes optional mechanisms without implying that every combination is available through every entry point.

The `workspace-v1` catalog entry accepts SQLite, Redis, and TiKV, but rejects etcd. Attaching a Packed lower additionally requires the catalog's permission, identifier, and lifecycle capabilities. The default `WorkspaceLower::Native` means native catalog layers; it is distinct from the `native_base` subsystem. Native P1 requires a workspace-v1 schema-1 base and a KV namespace. Its CLI explicitly rejects a frozen metadata volume rather than treating the mutable KV path as a fallback. Library support for frozen Native metadata therefore does not establish that this mount entry supports it.

Reader and Entity bounds. The standalone Packed entry accepts the current wire005 format through a content-addressed manifest; the CLI rejects earlier Packed v1/v2 en-

[1] Fixed sources: `https://github.com/brewfs/brewfs/tree/242af5f7e9bba7afc0c61778f1084fcb20e083a0` and `https://github.com/Ivanbeethoven/rustfs/tree/cc4926fac1107aee76540f28bf22dbd47fcbca24`.

Table 3: Current mount entry points and their construction. Compile-time features and backend capabilities further restrict admission.

Entry point	Constructed file-system view
Ordinary `flat-v1`	Ordinary metadata store and VFS with an object-data path; no workspace upper/lower construction.
`workspace-v1`	One sealed catalog base and one private writable upper; an optional persisted binding selects a Packed v3 lower adapter.
Standalone Packed v3	Authenticated packed snapshot and read-only VFS; no private writable upper is required.
Native P1 (`workspace-native-v2`)	Workspace-v1 KV namespace and base-data source, with Native data runtime and ordinary VFS.
Experimental Entity (`workspace-entity-v1`)	Fresh Redis-backed current-entity namespace over a packed manifest, S3 data, and a dedicated FUSE adapter.

tries. It also rejects nonzero payload memory-cache, payload SSD-cache, and prefetch budgets. Supported metadata caches must be distinguished from these unavailable payload-cache configurations.

Entity is an experimental, fresh-only path with process-crash takeover disabled. Its adapter uses direct I/O, zero metadata TTL, and no ordinary VFS slice-cache or writeback path. The Entity data API bounds an individual read, write, and logical chunk size at 16 MiB; the FUSE adapter separately limits writes to 1 MiB. Its complete-directory resolver bounds enumeration at 4,096 entries. Durable Entity lower ownership is limited to 256 active owner records per catalog store, with admission and census checks. This is distinct from the separate bounded set of Packed reader leases. These are path-specific constraints, not global file-size, directory-size, or workspace limits for all mounts. Entity upper reads currently fetch and authenticate the complete stored object before returning the requested subrange.

Publication and recovery. The code includes authenticated snapshot production, persisted lower bindings, reader pins, and guarded publication. It also wires a restricted Packed recovery command for reusing the same persistent volume claim (PVC). That path checks recovery generation, lease and pod identities, and scoped metadata/PVC authority. It does not provide Entity takeover or automatic migration to an arbitrary worker.

Native seal, publish, and fork functions encode quiescence, object verification, retention, and atomic revision transitions. The inspected tree contains these lifecycle definitions and unit-test callers, but no ordinary CLI handoff driver. Consequently, their protocol structure does not demonstrate an integrated worker transfer. Recovering a published file state also does not recover process memory or open application sessions; applications must establish suitable checkpoints and reopen their tools.

Object formation and layout gates. The ordinary object path fixes client block size g in configuration, with a default of 4 MiB inside 64 MiB logical chunks. Readers use the declared geometry, and incompatible volume configuration is rejected. There is no implemented access-driven controller selecting the proposed 1/4/8/16 MiB client sizes. Changing g on existing data requires preserving its interpretation, not simply changing a mount option.

The encoding-block-size hint requires an explicit compile-time feature, is off by default, and requires a compatible S3 backend. It does not change g or multipart part size. The inspected backend independently gates acceptance of B hints and creation of new q framing, each with enable and fleet-confirmation gates that default to off. Eligibility and geometry checks still apply. Reads and repair use the object's persisted layout; enabling a client hint does not re-encode existing objects.

Remaining connections and experimental controls. The ordinary uploader propagates configured hints, but the Entity mount constructs its S3 client with the default hint setting, and its strict durable create-only upload does not attach hint metadata. The Packed fixture producer also uses the default setting. Extending these paths must preserve conditional creation and precondition-failure behavior. Moreover, Entity directly uploads segments of each FUSE write rather than aggregating ordinary client blocks. Its upper objects therefore remain at most 1 MiB on this path and do not trigger the automatic hint policy.

That automatic hint policy omits a hint for $L_o \leq 1\,\text{MiB}$ and requests $B = 1\,\text{MiB}$ otherwise. The inspected research backend already defaults to $B = 1\,\text{MiB}$, so enabling the policy can leave effective B unchanged. Hint configuration also changes the ordinary client's fetch and cache policy, even for objects receiving no hint. Experiments must therefore record actual (g, L_o, B, q), eligibility, cache behavior, and physical read bytes, and isolate these factors. A configuration named *all-on* alone proves neither a complete combined runtime nor a layout treatment.

8 Evaluation Plan

This section is a protocol for future experiments. It contains no generated measurements or assumed speedups. The companion experiment matrix defines E00–E16, including instruments, negative cases, and execution gates. Table 4 connects the principal questions to those experiments. The existing passthrough observations in Section 2.2 supply a measured request-structure pilot for RQ1; they do not complete E01's locality/sharing study or any system-performance comparison.

8.1 Environment and baseline contracts

Our proposed primary testbed has two client machines, one dedicated metadata machine, and four RustFS storage machines connected by 25 GbE. A practical target is 24–32 physical cores and 128 GiB memory per client, with independent NVMe storage and fixed cache allocations. The initial storage geometry is one drive per storage host in a verified 2+2 set. These are requested resources, not a report of machines already used. A single-host four-drive setup serves only as a functional and mechanism testbed; it cannot establish independence of machine failure domains.

We will record processor, memory, disks, network, OS/kernel, FUSE settings, toolchain, complete configuration, build features, immutable image digests, and source revisions. Client object-block granularity g, durability, compression, cache budget, readahead, direct I/O, metadata TTL, inline thresholds, and backend gates are part of the treatment definition. Larger coding geometries will be separate experiments because they can change whether the selected q is smaller than the logical shard size.

External baselines include full copying, reflinks where supported, OverlayFS, BranchFS, and an object-backed file system such as JuiceFS [7] sharing the same backend resources. At least one mature immutable lower, such as Nydus or CernVM-FS, will be tested with a compatible writable layer. An immutable sorted-KV lower with equal records and cache capacity provides a specific layout control. A real DeltaFS comparison depends on its operation and workflow compatibility; we will not label our sorted-KV control "DeltaFS."

We will compare local workspace service separately from remote durable service. An OverlayFS local fsync is not equivalent to remote coded-object completion. Similarly, BranchFS's ephemeral branch uses different fsync and commit semantics [20]. Where a baseline needs a checkpoint to meet the selected persistence boundary, its completion cost is included. If equivalent semantics cannot be provided, we report separate operating points instead of one unconditional speed ranking.

8.2 Workloads and measurements

RQ1. The workload study will collect repository navigation, reading, editing, build/test, and cleanup activity from a pinned agent harness and model configuration. It will measure read-set overlap, directory locality, reuse distance, mutation footprint, and per-task operation distributions. Expected lookup and attribute probes are distinguished from I/O failures. An instrumentation-on/off control will quantify trace overhead and loss. The pilot motivates retaining Core and Exploratory task strata, both successful and negative metadata requests, mixed request sizes, and temporal bursts. Its aggregate method counts and size histograms are insufficient for semantics-preserving replay: the next capture must retain paths, payloads, handle dependencies, synchronization relations, and tool-phase boundaries. It must also connect FUSE request sizes to client object-block granularity g, final uploaded object lengths L_o, and actual client fetches before testing (B, q) policies. These observations distinguish object formation from request selection and backend processing.

The proposed expanded panel contains 20 compatible tasks each from SWE-bench Verified, a fixed public SWE-bench Pro version [19], and a multilingual set informed by Multi-SWE-bench [6, 25]. Exact task lists and revisions will be frozen after capability screening. An official audit has identified limitations in using SWE-bench Verified to measure frontier model ability [15]; we use it as a reproducible workload source and retain other task families. Filesystem performance does not establish improved reasoning ability.

The minimum live-agent core will use 20 prespecified tasks, three systems, and three independent repetitions at one concurrency setting: 180 runs. The 60-task extension requires 540 runs for that same setting, before any additional model or concurrency sweep. Pilot task duration and token use will determine a frozen budget. Broad concurrency sweeps first use replay; they are not silently multiplied across every live condition.

To make the file-system load explicit, each task is segmented into six traceable phases: (A) workspace creation and dependency discovery, (B) repository navigation ('stat', 'readdir', 'open', 'readlink', and small reads), (C) edit and patch writes, (D) build and test with sequential reads and generated files, (E) dependency installation or cleanup with create, rename, unlink, and hard-link operations, and (F) diff, discard, or retained workspace teardown. We retain the phase boundaries in the trace and report both per-phase and whole-task critical paths. A controlled replay uses the same paths, payload hashes, handle lifetimes, and synchronization events; the live run additionally records model/tool spans. This prevents a fast provisioning path from being mistaken for a faster build, and exposes the small-file metadata pressure that a fio-only study would miss.

The primary agent-load panel is a small, prespecified matrix rather than a post-hoc selection of favorable traces:

RQ	Question	Primary evidence	Plan
1	Which workload properties favor shared immutable state?	Observed: per-task FUSE request mix, read/write sizes, and time variation. To measure: cross-task read-set overlap, mutation footprint, locality, and tool-phase-labelled access.	E01–E02
2	When do delta-based workspaces reduce workspace cost?	Time to usable workspace, private metadata, modification and lifecycle costs	E03–E04
3	What does packed metadata save, and at what cost?	Metadata-KV calls, object requests/bytes, latency, cache effects, construction amortization	E05–E07
4	What changes with g, (B,q), and client read policy?	Object formation, persisted geometry, physical bytes, verification/EC CPU, durable writes and repair	E08–E10
5	Do the mechanisms provide additional combined value?	Legal factorial or nested comparisons under one runtime contract	E11
6	Do the effects matter for complete multi-agent tasks?	Deterministic replay, live-task makespan, resource use, output-quality controls	E12–E13
7	Which semantics and failure boundaries hold?	Differential operations, fault injection, integrity, reclamation and long-run resource bounds	E14–E16

Table 4: Research questions and evidence status. RQ1 separates the observed request-structure pilot from further measurements; the remaining rows are planned. E00 first checks that each configuration is connected and produces the claimed effective treatment.

Phase	Representative operations	Treatment and baseline controls	Primary outputs
A/B	fan-out, path walk, 'git status', grep, metadata-only scans	shared lower versus materialized checkout; cold/index-warm/data-warm states	ready time, lower-KV calls, GETs, metadata P99
C	patch, overwrite, truncate, sparse write, fsync	matched write-completion and cache budgets; 0.1–10% mutation	upper bytes, commit and drain tail, write amplification
D	compiler reads, test binaries, generated artifacts	same dependency image and local build cache; sequential/random mix	critical path, read bytes, CPU, task makespan
E/F	install, rename/unlink storms, retry, diff/discard	same lifecycle and durability boundary; retain failed and timed-out runs	whiteouts, retained objects, cleanup lag, success

Table 5: Agent file-system load panel. The table defines the workload phases; it does not assert that every agent task exercises them equally.

RQ2. Provisioning experiments vary lower size, workspace count, and modification fraction. Initial values are 10^4–10^6 files, 1–128 workspaces, and 0–50% modified files. We measure control-plane return, mount readiness, first navigation, and first durable edit separately. Prepositioned and cold-staged baselines are separate cases. Churn experiments add deletion, rename, truncation, and repeated lifecycles, subject to the selected implementation's ownership and cleanup support.

RQ3. Metadata lookup experiments compare mutable per-file KV, immutable sorted-KV, and the packed layout using equivalent values and cache budgets. Standalone read-only access tests the packed representation independently of delta-based workspaces; workspace experiments then compare the same representations as alternative lowers. Both include positive and negative lookup, direct getattr, enumeration, lookup-plus-read, deep paths, and wide directories. The snapshot reader's large-directory study is separated from the Entity bridge's 4,096-entry limit. We count upper/control/lower KV requests independently, as well as index, GroupMeta, descriptor, and payload object requests and bytes.

Cold-client, index-warm, metadata-warm, and data-warm states are explicitly defined; backend page-cache state is another variable. Unsupported payload-cache controls are not treated as available flags. Snapshot construction, publication, rebuilding, and reclamation enter each resource accounting metric. We report a break-even reuse count only if the measured net benefit per reuse is positive, with the reuse unit fixed for each standalone or workspace experiment.

RQ4. A backend-only study uses a fixed S3 request stream before adding the file-system path. Object sizes cross inline, 1 MiB, encoding-block, and multipart boundaries. B settings include default, the current automatic hint policy, and the four supported fixed values. q settings are legacy or the implemented selector, grouped by *persisted* geometry. If automatic and default produce the same (B,q), they are a no-treatment control, not evidence for a benefit from the automatic hint policy.

The file-system study adds g as a separate object-formation factor. A controlled sweep holds the logical write stream constant and records final object lengths L_o, object counts, and uploaded bytes, including compression, headers, and tails. It first holds backend geometry fixed, so changes caused by forming different objects are not attributed to B or q. Subsequent comparisons cross these factors only where the selected upload path supports them; a fixed configuration does not establish an automatic selection policy. The ordinary client's range-read threshold scales with g. A causal sweep must hold the absolute threshold constant where sup-

ported, or expose its change as a coupled client-policy factor.

Reads vary length, alignment, crossing of encoding-block and q-interval boundaries, and healthy versus degraded state. The primary measurements are physical data/parity read bytes, checksum work, EC CPU, HTTP payload, request counts, and latency. Client range-read and whole-object fetch policies are then separated while keeping cache admission and coalescing constant or exposing them as additional factors. Upper and lower results are separate because their integrity and upload paths differ.

Writes include append, overwrite, tails, multipart retry, and frequent durability requests. We record accepted writes, staging, remote completion, metadata commit, and drained background debt. Repair experiments measure read/write/network work and interference with healthy requests. The paper will claim lower write amplification only if these measurements support it; a read benefit can coexist with increased checksum or write cost.

8.3 Composition and complete agent execution

RQ5. The mechanisms address separate choices: workspace view composition, frozen metadata representation, and object formation and layout. Their comparisons must retain one runtime contract. Switching from a buffered ordinary volume to a direct-I/O Entity mount is not a pure delta-based workspace ablation. We will implement or validate matched controls for complete metadata materialization versus shared lower state, metadata representation, g, and backend geometry, with client read policy held constant or varied explicitly. Packed metadata supports standalone read-only use and can also serve as an optional workspace lower; neither role makes delta-based workspaces a prerequisite for packing. Only configurations that can actually be constructed enter a factorial comparison. If a selected entry point wires a treatment only within a particular workspace path, its comparison is nested within that runtime because of the implementation connection. Where full crossing is possible, task-level log-latency interactions will test whether combined effects differ from independent component effects.

RQ6. Deterministic replay and live agents answer different questions. Replay freezes tool inputs, write contents, and dependencies, preserving file-tree and output checks. A FUSE log containing only inode IDs and request lengths is not necessarily replayable; paths, handle lifetimes, payload references, initial state, and ordering must be recovered or recaptured. Arrival-driven and closed-loop replay are reported separately.

Live AX/DSH experiments keep the task, model identifier, prompt, tools, token/time budgets, and concurrency policy fixed. Mount probes must verify that workspaces and subprocesses reach the intended file system rather than a fallback directory. We measure batch makespan, tasks per hour, provisioning, resource use, and the traced storage critical path. Model waiting and I/O can overlap, so their durations cannot simply be added. Output correctness is checked with the task harness; an exit code of zero is not a resolved issue.

8.4 Correctness, failures, and analysis

RQ7. Differential and known-answer tests cover whiteouts, same-name recreation, lower-file modifications, truncate/extend, holes, links, rename-overwrite, open-unlink-read, and cross-workspace visibility within the supported contract. Fault injection targets object upload, manifest and binding publication, mapping commit, process restart, missing objects, frame corruption, and repair interruption. Fresh-only Entity behavior is reported as such; reopen or crash-takeover experiments require a completed runtime extension. Fail-closed behavior is distinct from successful recovery.

Long runs cover multiple cache and cleanup cycles and track tail latency, memory, open descriptors, owners, pending uploads, retained objects, and reclamation backlog. Background publishing or repair is added only where the selected runtime supports it. High mutation, poor locality, frequent snapshot rebuilding, and large sequential reads are explicit boundary workloads, not exclusions from the evaluation.

The independent unit is a run or a task, not each of its millions of requests. Initial plans use at least ten independent microbenchmark blocks and three live repeats per selected task; the formal sample sizes are frozen after a variance pilot. Conditions are interleaved in randomized host/time blocks. We report absolute effects, paired ratios, and 95% task- or repository-clustered bootstrap intervals. Repeated task outcomes are not treated as independent binary pairs.

For live speed comparisons, the primary endpoint includes all assigned tasks under the same deadline; failed and timed-out attempts remain in the throughput and completion accounting. Completed-only latency is secondary and accompanied by success rates and failure causes. Early failures do not count as fast completions. This protocol prevents storage behavior or model failures from creating an apparent speedup through selective omission.

9 Related Work and Limits of the Claim

Workspaces and agent-native file systems. OverlayFS supplies the layered semantics that motivate the private upper and shared lower [12]. BranchFS provides copy-on-write branch contexts for agentic exploration, including a commit/abort lifecycle; its workshop prototype and proposed kernel abstraction must be distinguished [20]. YoloFS studies staging, snapshots, and permissions to improve visibility and control over agent effects [27]. AgentFS also documents a file-system and tool-state abstraction for agents [3]. These

efforts make an "agent filesystem" label insufficient as a novelty claim. BrewFS studies private workspace views, frozen metadata access, and object formation and layout as optional capabilities over an object-backed file system. It does not replace process isolation or provide all of these systems' user-control and commit semantics.

Table 6 makes the intended advantage more precise. The last column is a hypothesis to be tested under matched persistence and cache budgets, rather than a claim that the prototype dominates every row.

Namespace and metadata organization. TableFS, IndexFS, LocoFS, and InfiniFS demonstrate that record organization, partitioning, and caching can substantially change metadata access paths [11, 13, 17, 18]. Recent systems also revisit hierarchical object metadata and distributed update coordination [10, 23]. We compare mechanisms at their actual service boundaries instead of assuming that every baseline performs a remote per-file lookup.

DeltaFS is a particularly close conceptual predecessor. It uses private namespaces and immutable mutation logs over shared storage, with selective composition of prior state [26]. BrewFS cannot claim private deltas or object-resident snapshots as inventions. Its packed representation places common attributes and data locators in bounded access units reached through indexes. This representation supports a read-only view independently and can also supply the immutable lower of an online private workspace. Conversely, a delta-based workspace can use a native metadata lower. The relevant comparisons therefore separate view composition from metadata representation before testing their combined cost.

Immutable distribution and storage backends. CernVM-FS provides demand-driven software distribution with indexed catalogs, while Nydus describes metadata/data organization for lazy image access [2, 14]. Slacker and DADI also address image distribution and startup costs [4, 9]. These are stronger points of comparison for immutable baselines than a model that downloads every file before use. The experiment plan therefore requires a mature lazy-lower baseline and counts image construction and staging separately. Packing must justify its access and construction costs in standalone read-only use as well as reuse across writable workspaces.

Distributed systems such as Ceph and Tectonic separate aspects of metadata and data services [16, 22]; this separation itself is not a contribution of BrewFS. Erasure-coded storage research considers repair cost, changing redundancy over a file's lifetime, and the relation between data placement and redundancy [1, 5, 8]. Okapi's separation of striping and redundancy grouping is different from our (B,q) distinction, although both motivate making coupled decisions explicit. Client block granularity g additionally determines how file data forms objects; it is distinct from backend coding and physical verification granularity. Neither forming data blocks nor choosing coding layouts is itself a new principle. We keep code rate fixed and measure object counts, checksum, tail, and repair cost rather than inferring a lower redundancy ratio from smaller blocks. These storage choices apply to ordinary volumes as well as workspace data and packed snapshots where the selected object path supports them.

Scope and unresolved evidence. Each capability has its own useful region. Sparse private changes favor sharing a baseline; repeated access to frozen records can amortize packing; object formation and backend layout depend on the read/write mix and physical processing cost. Poor name locality, high mutation, or rapid snapshot churn may favor existing mutable metadata services or local overlays without ruling out a useful storage-layout choice. The current entry points and lifecycle support constrain which combinations can be tested, as described in Section 7. Results must establish the benefit of each enabled mechanism and any additional benefit of composition; enabling all three is not itself evidence of a gain.

10 Conclusion

BrewFS organizes three optional capabilities over a common file-system and object-storage path. Delta-based workspaces compose a shared baseline with private changes. Packed metadata provides indexed access to frozen records for standalone read-only views or workspace lowers. Variable-granularity object storage separates client object formation at g from backend encoding and physical verification at (B,q). Each capability can be enabled for its own purpose; none requires the other two as an architectural prerequisite. The planned evaluation will compare supported configurations under matched contracts to determine their individual and combined costs, their effects on complete agent execution, and where existing approaches remain preferable. Composition does not imply an additional gain, and no quantitative performance conclusion is drawn in this draft.

References

[1] Sanjith Athlur, Timothy Kim, Saurabh Kadekodi, Francisco Maturana, Xavier Ramos, Arif Merchant, K. V. Rashmi, and Gregory R. Ganger. Okapi: Decoupling data striping and redundancy grouping in cluster file systems. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 897–914. USENIX Association, 2025.

[2] Jakob Blomer, Carlos Aguado-Sánchez, Predrag Buncic, and Artem Harutyunyan. Distributing LHC ap-

System	Shared baseline	Private view	Remote metadata/data	Boundary tested in this work
OverlayFS	local lower tree	kernel upper/copy-up	none by itself	remote immutable lower, explicit object reads, and durable upper
DeltaFS	immutable logs and manifests	per-job namespace	object store plus registry	current-value upper, directory-oriented pages, and online POSIX lookup
BranchFS	local COW branch context	branch commit/abort	prototype-local	remote persistence and multi-client lower sharing
CVMFS/Nydus	lazy immutable catalog/image	requires an external upper	cached objects/blobs	writable workspace semantics and metadata/data locator co-location
JuiceFS	chunk/slice/block volume	shared mutable POSIX view	object store plus KV	frozen metadata access, object formation at g, and persisted (B,q) layout
BrewFS	native or packed baseline	optional private delta	objects plus metadata/control KV	separate and combined effects of view composition, packing, and object layout

Table 6: Comparison axes used to choose baselines. Existing systems already provide individual mechanisms; the opportunity examined here is their independent use and optional composition for remote agent workloads.

plication software and conditions databases using the CernVM file system. *Journal of Physics: Conference Series*, 331(4):042003, 2011.

[3] Pekka Enberg and Glauber Costa. The missing abstraction for AI agents: The agent filesystem. Turso engineering blog, November 2025. Published 2025-11-13; official software announcement, not a peer-reviewed paper.

[4] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195. USENIX Association, 2016.

[5] Cheng Huang, Huseyin Simitci, Yikang Xu, Aaron Ogus, Brad Calder, Parikshit Gopalan, Jin Li, and Sergey Yekhanin. Erasure coding in Windows Azure Storage. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, pages 15–26. USENIX Association, 2012.

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[7] Juicedata, Inc. JuiceFS architecture. JuiceFS Community Edition documentation. Official technical documentation; page last updated 2025-11-06, accessed 2026-10-22.

[8] Timothy Kim, Sanjith Athlur, Saurabh Kadekodi, Francisco Maturana, Dax Delvira, Arif Merchant, Gregory R. Ganger, and K. V. Rashmi. Morph: Efficient file-lifetime redundancy management for cluster file systems. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP '24)*, pages 330–346. Association for Computing Machinery, 2024.

[9] Huiba Li, Yifan Yuan, Rui Du, Kai Ma, Lanzheng Liu, and Windsor Hsu. DADI: Block-level image service for agile and elastic application deployment. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 727–740. USENIX Association, 2020.

[10] Jiahao Li, Biao Cao, Jielong Jian, Cheng Li, Sen Han, Yiduo Wang, Yufei Wu, Kang Chen, Zhihui Yin, Qiushi Chen, Jiwei Xiong, Jie Zhao, Fengyuan Liu, Yan Xing, Liguo Duan, Miao Yu, Ran Zheng, Feng Wu, and Xianjun Meng. Mantle: Efficient hierarchical metadata management for cloud object storage services. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP '25)*, pages 928–943. Association for Computing Machinery, 2025.

[11] Siyang Li, Youyou Lu, Jiwu Shu, Yang Hu, and Tao Li. LocoFS: A loosely-coupled metadata service for distributed file systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*, 2017.

[12] Linux Kernel Contributors. Overlay filesystem. Linux kernel documentation. Official technical documentation, accessed 2026-10-22; pin the tested kernel and mount options in the evaluation.

[13] Wenhao Lv, Youyou Lu, Yiming Zhang, Peile Duan, and Jiwu Shu. InfiniFS: An efficient metadata service for large-scale distributed filesystems. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*, pages 313–328. USENIX Association, 2022.

[14] Nydus Authors. Registry accelerated file system. Nydus official documentation. Official technical documentation, accessed 2026-10-22; not a peer-reviewed paper.

[15] OpenAI. Why SWE-bench verified no longer measures frontier coding capabilities. OpenAI research an-

nouncement, February 2026. Published 2026-02-23; official benchmark audit, not a peer-reviewed paper.

[16] Satadru Pan, Theano Stavrinos, Yunqiao Zhang, Atul Sikaria, Pavel Zakharov, Abhinav Sharma, Shiva Shankar P, Mike Shuey, Richard Wareing, Monika Gangapuram, Guanglei Cao, Christian Preseau, Pratap Singh, Kestutis Patiejunas, JR Tipton, Ethan Katz-Bassett, and Wyatt Lloyd. Facebook's Tectonic filesystem: Efficiency from exascale. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 217–231. USENIX Association, 2021.

[17] Kai Ren and Garth Gibson. TABLEFS: Enhancing metadata efficiency in the local file system. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 145–156. USENIX Association, 2013.

[18] Kai Ren, Qing Zheng, Swapnil Patil, and Garth Gibson. IndexFS: Scaling file system metadata performance with stateless caching and bulk insertion. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14)*, 2014.

[19] Scale AI. SWE-bench pro v2. Official dataset card on Hugging Face, September 2026. Version 2.0.0, released 2026-09-22; official dataset, accessed 2026-10-22.

[20] Cong Wang and Yusheng Zheng. Fork, explore, commit: OS primitives for agentic exploration. In *Agentic OS Workshop at ASPLOS 2026*, March 2026. Workshop paper; arXiv:2602.08199v2. The proposed branch() syscall is not implemented in the evaluated prototype.

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[22] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 307–320. USENIX Association, 2006.

[23] Jingwei Xu, Mingkai Dong, Qiulin Tian, Ziyi Tian, Tong Xin, and Haibo Chen. SwitchFS: Asynchronous metadata updates for distributed filesystems with in-network coordination. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, 2026.

[24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[25] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Aoyan Li, Lu Chen, Xiaojian Zhong, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Ming Ding, and Liang Xiang. Multi-SWE-bench: A multilingual benchmark for issue resolving. In *Advances in Neural Information Processing Systems*, volume 38, 2025. Datasets and Benchmarks Track.

[26] Qing Zheng, Charles D. Cranor, Gregory R. Ganger, Garth A. Gibson, George Amvrosiadis, Bradley W. Settlemyer, and Gary Grider. DeltaFS: A scalable no-ground-truth filesystem for massively-parallel computing. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '21)*, 2021.

[27] Shawn Wanxiang Zhong, Junxuan Liao, Jing Liu, Mai Zheng, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Don't let AI agents YOLO your files: Information and control in agent-native filesystems. In *Proceedings of the 32nd ACM Symposium on Operating Systems Principles (SOSP '26)*. Association for Computing Machinery, 2026.